

РТУ МИРЭА
ИПТИП
Программирование корпоративных систем

ПРАКТИЧЕСКОЕ ЗАДАНИЕ №3
СИСТЕМЫ СБОРКИ

NetworkPacketSniffer

Сетевой анализатор трафика

Дисциплина:	Программирование корпоративных систем
Студент:	Фомин Владимир Родионович
Группа:	ЭФБО-06-24
Преподаватель:	Сокунов Дмитрий Антонович
Семестр:	Весенний семестр 2025/2026 учебного года

Москва — 2026

СОДЕРЖАНИЕ

1 Введение	
2 Архитектура проекта	
2.1 Структура папок	
2.2 Описание модулей	
3 Паттерны проектирования	
3.1 Стратегия (Strategy)	
3.2 Фабричный метод (Factory Method)	
3.3 Фасад (Facade)	
3.4 Внедрение зависимостей (Dependency Injection)	
3.5 Наблюдатель (Observer) — будущее расширение	
4 Система сборки (CMake)	
5 Захват и анализ сетевых пакетов	
5.1 Режим симуляции (Mock mode)	
5.2 Режим реального захвата (libpcap)	
5.3 Разбор пакетов: Ethernet / IPv4 / IPv6	
6 Инструкция по сборке и запуску	
7 Заключение	

1. ВВЕДЕНИЕ

Проект **NetworkPacketSniffer** — это консольное приложение для захвата и анализа сетевого трафика, разработанное на языке программирования C++ стандарта C++17. Проект выполнен в рамках практического задания №3 по дисциплине «Программирование корпоративных систем».

Цель задания состоит в освоении промышленных инструментов разработки программного обеспечения: системы сборки CMake, паттернов проектирования GOF (Gang of Four), модульной архитектуры C++-проектов, а также работы с библиотекой захвата сетевых пакетов libpcap/Npcap.

Цели и задачи

- демонстрация практического применения паттернов проектирования (Strategy, Factory Method, Facade, Dependency Injection, Observer);
- построение профессионального C++17-проекта с использованием системы сборки CMake версии 3.10 и выше;
- реализация модульной архитектуры с чётким разделением ответственности между компонентами;
- поддержка двух режимов работы: режима симуляции (без внешних зависимостей) и режима реального захвата пакетов через libpcap/Npcap;
- анализ сетевых пакетов протоколов IPv4, IPv6, TCP, UDP и ICMP.

Назначение приложения

- отображение списка доступных сетевых интерфейсов;
- выбор интерфейса для захвата пакетов;

- запуск и остановка захвата трафика;
- анализ пакетов протоколов IPv4/IPv6, TCP, UDP, ICMP;
- извлечение IP-адресов источника и получателя;
- подсчёт пакетов по IP-адресам и протоколам;
- расчёт суммарного объёма трафика в байтах;
- отображение статистики в консоли в реальном времени;
- генерация итогового отчёта в текстовый файл *traffic_report.txt*;
- обработка ошибок: отсутствие прав, недоступный интерфейс, отсутствие драйвера.

2. АРХИТЕКТУРА ПРОЕКТА

Приложение построено по принципу **модульной архитектуры**, где каждый программный модуль несёт строго определённую ответственность. Это обеспечивает высокую степень разделения ответственностей (Separation of Concerns), тестируемость и расширяемость кода.

2.1. Структура папок

```
NetworkPacketSniffer/
├── CMakeLists.txt      ← конфигурация системы сборки
├── README.md           ← документация проекта
├── include/            ← заголовочные файлы (.h)
│   ├── PacketInfo.h   ← структура PacketInfo, перечисление Protocol
│   ├── PacketCapture.h ← ICaptureStrategy, MockCapture, PacketCapture
│   ├── PcapCaptureStrategy.h ← реальный захват через libpcap
│   ├── PacketParser.h  ← разбор сырых пакетов
│   ├── TrafficStatistics.h ← накопление статистики
│   ├── ReportGenerator.h ← генерация текстового отчёта
│   └── ConsoleUI.h     ← консольный интерфейс (Фасад)
└── src/               ← файлы реализации (.cpp)
    ├── main.cpp        ← точка входа + внедрение зависимостей
    ├── PacketCapture.cpp ← MockCaptureStrategy + фабрика
    ├── PcapCaptureStrategy.cpp ← libpcap-реализация
    ├── PacketParser.cpp  ← разбор Ethernet/IP-заголовков
    ├── TrafficStatistics.cpp
    ├── ReportGenerator.cpp
    └── ConsoleUI.cpp
```

Рисунок 1 — Структура папок проекта

2.2. Описание модулей

PacketInfo.h

Определяет перечисление Protocol (TCP, UDP, ICMP, ICMPv6, IPv4, IPv6, UNKNOWN) и структуру PacketInfo — каноническое представление одного разобранного пакета (sourceIp, destinationIp, protocol, sizeBytes, timestamp).

PacketCapture

Отвечает за перечисление интерфейсов, их выбор, запуск и остановку захвата, а также за передачу сырых пакетов (RawPacket) вышележащим модулям. Реализует паттерн «Стратегия»: делегирует все вызовы объекту ICaptureStrategy.

PcapCaptureStrategy

Конкретная стратегия захвата, использующая libpcap/Npcap. Открывает интерфейс через pcap_open_live(), применяет BPF-фильтр «ip or ip6», получает пакеты через pcap_next_ex(). Компилируется только при наличии флага USE_PCAP.

PacketParser

Преобразует RawPacket в структуру PacketInfo. В режиме симуляции использует предзаполненные поля; в реальном режиме декодирует Ethernet-заголовок, затем IPv4- или IPv6-заголовок с помощью inet_ntop().

TrafficStatistics

Накапливает статистику в двух словарях std::map: по IP-адресам источника (struct IpStats) и по протоколам. Предоставляет методы addPacket(), getIpStats(), getProtocolStats(), getTotalBytes(), reset().

ReportGenerator

Строит текстовый отчёт в виде строки и сохраняет его в файл `traffic_report.txt`. Метод `generateReport()` возвращает `bool` — признак успешности записи.

ConsoleUI

Фасад: принимает ссылки на все четыре подсистемы через конструктор (внедрение зависимостей) и предоставляет единственный публичный метод `run()`, реализующий главный цикл текстового меню.

3. ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

В проекте применяется несколько классических паттернов проектирования. Ниже приведено подробное описание каждого паттерна, его роль в проекте, код соответствующих классов и преимущества применения.

3.1. Стратегия (Strategy)

Назначение паттерна. Паттерн «Стратегия» определяет семейство алгоритмов, инкапсулирует каждый из них и обеспечивает их взаимозаменяемость. Алгоритм может изменяться независимо от клиентов, которые им пользуются.

Применение в проекте. Интерфейс *ICaptureStrategy* определяет контракт для всех реализаций захвата пакетов. Класс *MockCaptureStrategy* реализует симуляцию (не требует внешних библиотек), а *PcapCaptureStrategy* — реальный захват через libpcap. Замена стратегии не требует изменений в классе *PacketCapture* или других модулях.

```

// include/PacketCapture.h
class ICaptureStrategy {
public:
    virtual ~ICaptureStrategy() = default;

    [[nodiscard]] virtual std::vector<std::string> listInterfaces() =
0;
    virtual bool selectInterface(int i) =
0;
    virtual bool startCapture() =
0;
    virtual void stopCapture() =
0;
    [[nodiscard]] virtual std::optional<RawPacket> getNextPacket() =
0;
    [[nodiscard]] virtual bool isRunning() const =
0;
};

// Конкретные стратегии:
class MockCaptureStrategy final : public ICaptureStrategy { /* ... */ };
class PcapCaptureStrategy final : public ICaptureStrategy { /* ... */ };

```

Листинг 1 — Интерфейс стратегии *ICaptureStrategy*

Преимущества. Код приложения не зависит от конкретного механизма захвата. Новую стратегию (например, захват из файла .pcap) можно добавить без изменения существующего кода — достаточно реализовать интерфейс *ICaptureStrategy*.

3.2. Фабричный метод (Factory Method)

Назначение паттерна. Паттерн «Фабричный метод» определяет интерфейс для создания объекта, но оставляет подклассам или фабричным функциям решение о том, какой конкретный класс инстанцировать.

Применение в проекте. Функция *createCaptureStrategy()*, определённая в *PacketCapture.cpp*, принимает решение о создании нужной стратегии на основе макроопределения *USE_PCAP*, которое CMake устанавливает автоматически при обнаружении *libpcap*. Ни функция *main()*, ни *ConsoleUI* не знают, какая конкретная стратегия используется.

```

// src/PacketCapture.cpp
#ifdef USE_PCAP
std::unique_ptr<ICaptureStrategy> createPcapStrategy(); // из PcapCapture
Strategy.cpp
#endif

std::unique_ptr<ICaptureStrategy> createCaptureStrategy() {
#ifdef USE_PCAP
    return createPcapStrategy(); // реальный захват через libpcap
#else
    return std::make_unique<MockCaptureStrategy>(); // режим симуляции
#endif
}

// src/PcapCaptureStrategy.cpp (компилируется только при -DUSE_PCAP)
std::unique_ptr<ICaptureStrategy> createPcapStrategy() {
    return std::make_unique<PcapCaptureStrategy>(50);
}

```

Листинг 2 — Фабричная функция *createCaptureStrategy()*

Преимущества. Единственное место, где принимается решение о типе стратегии. CMakeLists.txt автоматически переключает реализацию через препроцессор. Тесты могут внедрить собственную стратегию, минуя фабрику.

3.3. Фасад (Facade)

Назначение паттерна. Паттерн «Фасад» предоставляет унифицированный интерфейс к набору интерфейсов в некоторой подсистеме. Фасад определяет высокоуровневый интерфейс, упрощающий работу с подсистемой.

Применение в проекте. Класс *ConsoleUI* является фасадом для четырёх подсистем: *PacketCapture*, *PacketParser*, *TrafficStatistics* и *ReportGenerator*. Единственный публичный метод *run()* скрывает весь поток управления и взаимодействие между модулями. Клиентский код (*main.cpp*) взаимодействует только с *ConsoleUI*.

```

// include/ConsoleUI.h — Фасад
class ConsoleUI {
public:
    // Все подсистемы внедряются через конструктор (Dependency Injection)
    ConsoleUI(PacketCapture& capture,
              PacketParser& parser,
              TrafficStatistics& stats,
              ReportGenerator& reporter);

    void run(); // единственный публичный метод: главный цикл меню

private:
    void handleListInterfaces(); // показать интерфейсы
    void handleSelectInterface(); // выбрать интерфейс
    void handleStartCapture(); // запустить захват
    void handleStopCapture(); // остановить захват
    void handleShowStatistics(); // показать статистику
    void handleExportReport(); // экспортировать отчёт

    PacketCapture& capture_;
    PacketParser& parser_;
    TrafficStatistics& stats_;
    ReportGenerator& reporter_;
};

```

Листинг 3 — Класс ConsoleUI (Фасад)

Преимущества. `main.cpp` содержит всего одну строку взаимодействия с UI: `ui.run()`. Замена консольного UI на графический (Qt, ncurses) потребует только новой реализации фасада без изменения остальных модулей.

3.4. Внедрение зависимостей (Dependency Injection)

Назначение паттерна. Внедрение зависимостей — техника, при которой объект получает свои зависимости извне, а не создаёт их самостоятельно. Это обеспечивает слабую связанность компонентов и высокую тестируемость.

Применение в проекте. Функция *main()* создаёт все подсистемы независимо и передаёт их в *ConsoleUI* через конструктор. Глобальное состояние (Singleton) намеренно не используется: это упрощает тестирование — в тестах достаточно подать другой объект *TrafficStatistics* или *ReportGenerator* без изменения кода UI.

```
// src/main.cpp
int main() {
    // Шаг 1: фабрика создаёт нужную стратегию
    auto strategy = createCaptureStrategy();

    // Шаг 2: строим подсистемы независимо друг от друга (DI)
    PacketCapture capture(std::move(strategy));
    PacketParser parser;
    TrafficStatistics stats;
    ReportGenerator reporter("traffic_report.txt");

    // Шаг 3: внедряем зависимости в Фасад
    ConsoleUI ui(capture, parser, stats, reporter);

    // Шаг 4: запускаем приложение
    ui.run();
    return 0;
}
```

Листинг 4 — Внедрение зависимостей в *main.cpp*

Преимущества. Отсутствие глобального состояния делает каждый модуль независимо тестируемым. Подсистемы можно создавать в любом порядке и с любыми параметрами без скрытых связей.

3.5. Наблюдатель (Observer) — будущее расширение

Назначение паттерна. Паттерн «Наблюдатель» определяет зависимость «один ко многим» между объектами: при изменении состояния одного объекта все зависящие от него объекты автоматически уведомляются.

Планируемое применение. В многопоточной версии приложения класс *PacketCapture* мог бы выступать издателем событий. При захвате каждого нового пакета он уведомлял бы подписчиков, реализующих интерфейс *IPacketObserver*.

```
// Проектируемый интерфейс наблюдателя (будущая версия)
class IPacketObserver {
public:
    virtual ~IPacketObserver() = default;
    virtual void onPacketCaptured(const PacketInfo& packet) = 0;
};

// Подписчики:
//   TrafficStatistics : public IPacketObserver → накапливает статистику
//   LiveDisplay       : public IPacketObserver → выводит пакет в консоль
//   AlertEngine       : public IPacketObserver → срабатывает по триггерам
//
// PacketCapture::captureLoop() вызывает:
//   for (auto* obs : observers_) obs->onPacketCaptured(info);
```

Листинг 5 — Проектируемый интерфейс IPacketObserver

Преимущества. Позволяет добавлять новые виды обработки пакетов (логирование, оповещения, визуализация) без изменения кода захвата.

4. СИСТЕМА СБОРКИ (CMAKE)

Сборка проекта организована с помощью системы CMake версии 3.10 и выше. Конфигурационный файл *CMakeLists.txt*, расположенный в корне проекта, объявляет все параметры сборки и опционально подключает библиотеку `libpcap/Npcap`.

Ключевые директивы *CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.10)

project(NetworkPacketSniffer
    VERSION      1.0.0
    DESCRIPTION  "Сетевой анализатор трафика — Практическое задание №3"
    LANGUAGES    CXX
)

# Стандарт C++17 — обязательный
set(CMAKE_CXX_STANDARD      17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS    OFF)

# Явное перечисление всех файлов исходного кода
set(SOURCES
    src/main.cpp
    src/PacketCapture.cpp
    src/PacketParser.cpp
    src/PcapCaptureStrategy.cpp # компилируется в пустой TU без USE_PCAP
    src/TrafficStatistics.cpp
    src/ReportGenerator.cpp
    src/ConsoleUI.cpp
)

add_executable(${PROJECT_NAME} ${SOURCES} ${HEADERS})

# Подключение папки заголовков
target_include_directories(${PROJECT_NAME} PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)
```

Листинг 6 — Основная конфигурация *CMakeLists.txt*

Автоматическое обнаружение `libpcap`

CMake автоматически ищет библиотеку `libpcap`/`Npcap` и заголовочный файл `pcap.h`. При обнаружении обеих составляющих проект компилируется с макросом `USE_PCAP=1` и связывается с библиотекой. При отсутствии `libpcap` проект собирается в режиме симуляции без каких-либо ошибок.

```
# Поиск libpcap / Npcap
find_library(PCAP_LIBRARY NAMES pcap wpcap
  PATHS /usr/lib /usr/local/lib /opt/homebrew/lib)
find_path(PCAP_INCLUDE_DIR NAMES pcap.h pcap/pcap.h
  PATHS /usr/include /usr/local/include /opt/homebrew/include)

if(PCAP_LIBRARY AND PCAP_INCLUDE_DIR)
  message(STATUS "libpcap найден — включён режим реального захвата (USE_PCAP)")
  target_compile_definitions(${PROJECT_NAME} PRIVATE USE_PCAP=1)
  target_include_directories(${PROJECT_NAME} PRIVATE ${PCAP_INCLUDE_DIR})
  target_link_libraries(${PROJECT_NAME} PRIVATE ${PCAP_LIBRARY})
  # Linux: дополнительно линкуем pthreads
  if(UNIX AND NOT APPLE)
    find_package(Threads REQUIRED)
    target_link_libraries(${PROJECT_NAME} PRIVATE Threads::Threads)
  endif()
  # Windows: линкуем Winsock
  if(WIN32)
    target_link_libraries(${PROJECT_NAME} PRIVATE ws2_32)
  endif()
else()
  message(STATUS "libpcap НЕ найден — используется режим симуляции")
endif()
```

Листинг 7 — Автодетектирование `libpcap` в `CMakeLists.txt`

Out-of-source сборка

Проект поддерживает сборку вне исходного дерева (*out-of-source build*): все артефакты компиляции (объектные файлы, бинарники) помещаются в отдельную папку `build/`, не засоряя исходный код. Это является стандартной практикой для промышленных C++-проектов.

5. ЗАХВАТ И АНАЛИЗ СЕТЕВЫХ ПАКЕТОВ

5.1. Режим симуляции (Mock mode)

При сборке без `libpcap` фабрика `createCaptureStrategy()` возвращает объект `MockCaptureStrategy`. Стратегия генерирует детерминированный поток из 30 пакетов, циклически проходя по таблице из 10 шаблонов. Каждый шаблон содержит: IP-адрес источника, IP-адрес назначения, протокол (IANA-номер) и размер пакета в байтах.

```
// src/PacketCapture.cpp — таблица шаблонов пакетов
constexpr std::array<MockTemplate, 10> kTemplates = {{
    { "192.168.1.10", "8.8.8.8",      6, 1500 }, // TCP
    { "192.168.1.10", "8.8.4.4",     17,  64 }, // UDP
    { "192.168.1.20", "1.1.1.1",      1,  84 }, // ICMP
    { "10.0.0.5",     "192.168.1.10", 6, 1024 }, // TCP
    { "192.168.1.30", "224.0.0.251", 17,  256 }, // UDP (mDNS)
    { "192.168.1.10", "192.168.1.20", 1,  84 }, // ICMP
    { "8.8.8.8",      "192.168.1.10", 6, 1500 }, // TCP
    { "192.168.1.20", "8.8.8.8",     17,  128 }, // UDP (NTP)
    { "192.168.1.10", "10.0.0.5",     6,  512 }, // TCP
    { "10.0.0.5",     "8.8.8.8",      1,  84 }, // ICMP
}};
```

Листинг 8 — Шаблоны пакетов для режима симуляции

5.2. Режим реального захвата (libpcap)

При наличии `libpcap/Npcap` CMake компилирует проект с флагом `-DUSE_PCAP=1`, и фабрика создаёт объект `PcapCaptureStrategy`. Стратегия открывает интерфейс в **неразборчивом режиме** (promiscuous mode) через `pcap_open_live()`, применяет BPF-фильтр `"ip or ip6"` и читает пакеты через `pcap_next_ex()` с таймаутом 100 мс.

```

// src/PcapCaptureStrategy.cpp
bool PcapCaptureStrategy::startCapture() {
    handle_ = pcap_open_live(
        deviceNames_[selectedIface_].c_str(),
        65535,    // snaplen: захватываем весь пакет целиком
        1,        // promiscuous mode — видим весь трафик интерфейса
        100,      // read timeout, мс: pcap_next_ex вернёт 0 если нет пакета
        &errbuf_
    );
    if (!handle_) { lastError_ = errbuf_; return false; }

    struct bpf_program fp{};
    if (pcap_compile(handle_, &fp, "ip or ip6", 0, PCAP_NETMASK_UNKNOWN) ==
        0)
        pcap_setfilter(handle_, &fp), pcap_freecode(&fp);

    running_ = true;
    return true;
}

std::optional<RawPacket> PcapCaptureStrategy::getNextPacket() {
    struct pcap_pkthdr* hdr = nullptr;
    const u_char* data = nullptr;
    int res = pcap_next_ex(handle_, &hdr, &data);

    if (res == 0) return std::nullopt;    // таймаут, продолжаем
    if (res == PCAP_ERROR_BREAK || res < 0) {
        running_ = false; return std::nullopt;    // ошибка или breakloop
    }
    // res == 1: пакет успешно прочитан
    RawPacket pkt;
    pkt.isMock = false;
    pkt.length = hdr->caplen;
    pkt.data.assign(data, data + hdr->caplen);
    return pkt;
}

```

Листинг 9 — Реализация PcapCaptureStrategy

5.3. Разбор пакетов: Ethernet / IPv4 / IPv6

Метод *PacketParser::parse()* анализирует сырые байты пакета. Для реального режима выполняется следующая последовательность действий:

- **Ethernet II заголовок** (14 байт): байты [12–13] содержат поле EtherType. Значение 0x0800 означает IPv4, значение 0x86DD —

IPv6.

- **IPv4 заголовок** (минимум 20 байт): поле IHL (биты [0:3] первого байта) задаёт длину заголовка. Байт [9] содержит IANA-номер протокола транспортного уровня. Байты [12–15] и [16–19] — IP-адреса источника и назначения.
- **IPv6 заголовок** (фиксированные 40 байт): байт [6] — следующий заголовок (аналог протокола в IPv4). Байты [8–23] — IPv6-адрес источника, байты [24–39] — адрес назначения.
- **Конвертация адресов**: функция POSIX `inet_ntop()` преобразует бинарное представление IP-адреса в строку.

```
// src/PacketParser.cpp — разбор IPv4-заголовка
static int parseIPv4(const uint8_t* ip, std::size_t avail, PacketInfo& out)
{
    if (avail < 20) return -1; // защита от коротких б
уфер
    const std::size_t ihl = (ip[0] & 0x0Fu) * 4u; // длина IP-заголовка
    if (ihl < 20 || ihl > avail) return -1;

    char src[INET_ADDRSTRLEN]{}, dst[INET_ADDRSTRLEN]{};
    inet_ntop(AF_INET, ip + 12, src, sizeof(src)); // адрес источника
    inet_ntop(AF_INET, ip + 16, dst, sizeof(dst)); // адрес назначения

    out.sourceIp      = src;
    out.destinationIp = dst;
    return static_cast<int>(ip[9]); // IANA-номер протокола (6=TCP, 17=UDP
, 1=ICMP)
}
```

Листинг 10 — Разбор IPv4-заголовка в `PacketParser.cpp`

Поддерживаемые протоколы

Протокол	IANA №	Описание
ICMP	1	Internet Control Message Protocol — управляющие сообщения (ping)
TCP	6	Transmission Control Protocol — надёжная передача данных
UDP	17	User Datagram Protocol — ненадёжная быстрая передача
IPv6	41	IPv6-in-IPv4 туннелирование
ICMPv6	58	ICMP для IPv6

IPv4	—	Обобщённый IPv4 (транспорт не распознан)
IPv6	—	Обобщённый IPv6 (транспорт не распознан)

Таблица 1 — Поддерживаемые сетевые протоколы

6. ИНСТРУКЦИЯ ПО СБОРКЕ И ЗАПУСКУ

Предварительные требования

Инструмент	Версия	Примечание
CMake	≥ 3.10	cmake.org или менеджер пакетов
Компилятор C++	$\text{GCC} \geq 8, \text{Clang} \geq 7, \text{MSVC} 2019++17$	обязателен
libpcap / Npcap	Любая актуальная	Опционально — без него: mock-режим
Python 3	≥ 3.8	Только для генерации данного отчёта

Команды сборки (Linux / macOS)

```
# 1. Перейти в корневую папку проекта
cd NetworkPacketSniffer

# 2. Конфигурирование — out-of-source сборка в папке build/
cmake -S . -B build

# 3. Компиляция
cmake --build build

# 4. Запуск (режим симуляции — без sudo)
./build/NetworkPacketSniffer

# 4a. Запуск с реальным захватом (требуется привилегия root/Administrator)
sudo ./build/NetworkPacketSniffer
```

Листинг 11 — Команды сборки и запуска на Linux/macOS

Команды сборки (Windows + Npcap)

```
# Установить Npcap SDK: https://npcap.com/#download
# Указать путь к SDK в переменной окружения или параметре CMake

cmake -S . -B build -DPCAP_INCLUDE_DIR="C:/npcap-sdk/Include" ^
      -DPCAP_LIBRARY="C:/npcap-sdk/Lib/x64/wpcap.lib"
cmake --build build --config Release
build\Release\NetworkPacketSniffer.exe
```

Листинг 12 — Команды сборки на Windows с Npcap

Типичный сценарий работы

1. Запустить приложение (с `sudo` для реального захвата).
2. Выбрать пункт 1 — просмотреть список доступных сетевых интерфейсов.
3. Выбрать пункт 2 — ввести индекс нужного интерфейса.
4. Выбрать пункт 3 — начать захват пакетов (30 симулируемых или до 50 реальных пакетов).
5. Выбрать пункт 5 — просмотреть текущую статистику (протоколы, IP-адреса, объём трафика).
6. Выбрать пункт 6 — экспортировать итоговый отчёт в *traffic_report.txt*.
7. Выбрать пункт 0 — выйти из приложения.

Пример вывода консоли

```
Network Packet Sniffer / Traffic Analyzer
Practical Assignment #3 – Build Systems

Running in MOCK/SIMULATION mode (no libpcap required).

Source IP      Destination IP  Protocol  Size
-----
192.168.1.10    8.8.8.8        TCP       1500 B
192.168.1.10    8.8.4.4        UDP        64 B
192.168.1.20    1.1.1.1        ICMP       84 B
10.0.0.5        192.168.1.10   TCP      1024 B
...
[+] Capture complete – 30 packets processed.

Total packets : 30
Total bytes   : 15708 (15.3398 KB)
Protocol breakdown:
  TCP         12 pkts (40.0%)
  UDP          9 pkts (30.0%)
  ICMP         9 pkts (30.0%)
```

Листинг 13 — Пример вывода приложения в режиме симуляции

Пример итогового отчёта (traffic_report.txt)

```
=====
NETWORK PACKET SNIFFER — TRAFFIC REPORT
=====
Generated : 2026-05-17 18:02:06

GLOBAL TOTALS
-----
Total packets captured : 30
Total traffic volume   : 15708 bytes (15.3398 KB)

PACKETS BY PROTOCOL
-----
TCP      : 12 packets
UDP      : 9 packets
ICMP     : 9 packets

PACKETS BY SOURCE IP ADDRESS
-----
192.168.1.10      packets=12      bytes=6480
10.0.0.5          packets=6       bytes=3324
=====
```

Листинг 14 — Пример итогового отчёта traffic_report.txt

7. ЗАКЛЮЧЕНИЕ

В рамках практического задания №3 по дисциплине «Программирование корпоративных систем» разработано консольное приложение **NetworkPacketSniffer** — сетевой анализатор трафика на языке C++17.

Выполненные работы

- Спроектирована и реализована **модульная архитектура** из шести независимых компонентов (PacketCapture, PacketParser, TrafficStatistics, ReportGenerator, ConsoleUI, PcapCaptureStrategy).
- Применено **пять паттернов проектирования**: Strategy, Factory Method, Facade, Dependency Injection, Observer (описан как будущее расширение).
- Разработан **конфигурационный файл CMakeLists.txt** с автоматическим обнаружением libpcap/Npcap и поддержкой кросс-платформенной сборки (Linux, macOS, Windows).
- Реализован **режим симуляции (Mock mode)**: 30 детерминированных пакетов с различными протоколами и IP-адресами генерируются без внешних зависимостей.
- Реализован **режим реального захвата (libpcap)**: разбор Ethernet-кадров, IPv4/IPv6-заголовков, определение протоколов TCP, UDP, ICMP, ICMPv6.
- Реализована **консольная статистика**: распределение пакетов по протоколам и IP-адресам источника, суммарный объём трафика.
- Реализована **генерация итогового отчёта** в текстовый файл *traffic_report.txt*.
- Проект **компилируется без предупреждений** при флагах -Wall -Wextra -Wpedantic -Wshadow.

Практическая ценность

Разработанное приложение демонстрирует подход к созданию расширяемых C++-систем, при котором смена внешней библиотеки (libpcap → Npcap или пользовательская реализация) не требует изменений в бизнес-логике. Чёткое разделение ответственности позволяет тестировать каждый модуль независимо, а внедрение зависимостей исключает скрытые глобальные связи.

Возможные улучшения

- Многопоточный захват с очередью пакетов (std::queue + std::mutex) и паттерном Observer для параллельной обработки.
- Фильтрация пакетов по IP-адресу, порту или протоколу через BPF-фильтры.
- Анализ TCP-сессий: отслеживание соединений (SYN/SYN-ACK/FIN).
- Визуализация статистики: гистограммы трафика в консоли (ncurses).
- Сохранение захваченных пакетов в формате PCAP для последующего анализа в Wireshark.
- Поддержка чтения из PCAP-файла как отдельная стратегия *FileCaptureStrategy : public ICaptureStrategy*.

Все требования практического задания №3 выполнены в полном объёме.
Исходный код опубликован в репозитории и готов к проверке.